

Sentence-BERT を用いた用途・予算別 PC 構成推薦システム 付録資料

学籍番号 24026111, 氏名佐藤信晃

2026 年 8 月 1 日

1 LLM の利用記録

本課題では, LLM を以下の目的で利用した。

- 課題文の要件整理と, 既存プロトタイプとの差分確認
- 文字化けおよび Python 構文エラーの修正
- 入力検証, キャッシュ, 性能評価処理の実装支援
- レポート本文の構成と表現の推敲

主な指示は「NLPreport.md の要件に沿って, 既存の PC パーツ推薦プロトタイプを完成させること」であった。LLM から得た提案はそのまま採用せず, Python の構文検査, 依存ライブラリのインポート, Sentence-BERT による実検索, 10 件の一括評価を実行して確認した。

1.1 対話記録

以下では, 実際に LLM へ入力した指示の内容を時系列で示す。対話時の短い口語表現は, 意図を変えない範囲でレポート用の文章に要約した。したがって, 以下の記述は逐語的な対話ログではなく, 指示内容の要約である。各指示に対する応答の要点, 実際に採用した内容, 確認方法も併記する。

対話 1: 課題要件と既存実装の確認

ユーザの指示内容 (要約)

「NLPreport.mdに記載された課題要件を確認し, 既存の PC パーツ推薦システムを課題に適合するプロトタイプへ修正する。」

LLM の応答・作業

課題文, 既存の app.py, pc_builds.csv, requirements.txt を読み, 課題が Sentence-BERT または gensim を必ず利用する NLP プロトタイプの設計・実装・評価であると整理した。既存コードは日本語の文字化けに加え, 文字列の引用符が途中で失われており, Python として実行できない状態であると指摘した。

採用内容と確認

PC 構成推薦という題材は課題条件を満たすため維持し, 文字化けした UI 文言と構文を修正した。python -m py_compile app.py による構文検査と, Streamlit, pandas, sentence-transformers, torch のインポート確認を行った。

対話 2：システムのゴールの明確化

ユーザの質問内容（要約）

「PC 構成推薦システムの対象ユーザ，入出力，Sentence-BERT の利用箇所，および性能評価の方法を整理する。」

LLM の応答

利用者が自然文で PC 用途と予算を入力すると，Sentence-BERT で用途の意味を比較し，予算内の PC 構成を推薦するシステムを完成形として説明した。また，プロトタイプの作成だけでなく，Top-1・Top-3 正解率による評価と，指定形式の本文 PDF および付録 PDF が必要であると整理した。

採用内容

この説明を基に，実装では「構成を検索」と「性能評価」の二つの画面を用意し，レポートでは課題設定，技術概要，設計，実装，考察の順で説明することにした。

対話 3：推薦処理と評価処理の改善

ユーザの指示内容（要約）

「Sentence-BERT を利用した推薦処理を実装し，プロトタイプの性能を比較可能な指標で評価できるようにする。」

LLM の提案

検索のたびに全候補をベクトル化すると処理が重複するため，モデル，CSV データ，全候補の埋め込みをキャッシュする構成が提案された。また，空の入力や予算内候補がない場合の処理，登録文をそのまま使わない 10 件の言い換え評価が追加された。

採用内容と確認

`st.cache_resource`でモデルとコーパス埋め込みを，`st.cache_data`で CSV をキャッシュした。Sentence-BERT モデルを実際に読み込み，「WQHD でゲームを快適に遊びたい」という入力に対して「WQHD ゲーム」が類似度 0.883 で 1 位になることを確認した。さらに 10 件の一括評価を実行し，Top-1，Top-3 とともに 10/10 件となった。

対話 4：レポートテンプレートへの適用

ユーザの指示内容（要約）

「`NLP_template_UTF8.tex`をレポートのテンプレートとして使用し，実装内容と評価結果をレポート用の文章にまとめる。」

LLM の応答・作業

テンプレートの指定 section 名を変更しないという課題要件を確認し，「課題設定」「技術概要」「システムの設計」「プロトタイプの実装」「最終的なシステムの考察」の五章で本文を作成した。Sentence-BERT の仕組み，コサイン類似度の数式，処理フロー，関数構成，評価方法，制約と改善案を追加した。

確認

参考文献には Sentence-BERT の原論文，使用モデルの Model Card，Sentence Transformers 公式文書を記載した。ソースコード全文を本文に載せず，推薦処理の中核部分だけを掲載した。

対話 5：LaTeX から PDF への変換

ユーザの指示内容（要約）

「作成した LaTeX 文書をコンパイルし，提出内容を確認できる PDF を生成する。」

LLM の応答・作業

実行環境に pLaTeX がなかったため，Tectonic をプロジェクト内へ導入した。元テンプレートの `jsarticle`と `dvipdfmx`指定は Tectonic の XeTeX 処理に直接対応しなかったため，本文内容を維持したまま `article`，`xeCJK`，Windows の日本語フォントを使う設定へ変更した。日付はテンプレートと同様に `\date{\today}`を使い，「年・月・日」の日本語表記になるよう再定義した。

確認

コンパイル終了コード、PDF ヘッダ、ファイル終端を確認し、正常な PDF ファイルが生成されたことを確認した。

対話 6：実行画面と結果の追加

ユーザの指示内容（要約）

「プロトタイプの実行画面をレポートへ掲載し、表示された推薦結果をレポート用の文章と表で説明する。」

LLM の応答・作業

利用者が用意した二枚の実行画面を本文の実装章へ掲載した。画面に表示された入力条件、三つの推薦候補、類似度、価格、予算残額を表として整理した。単に画像を貼るだけでなく、第1候補が入力用途と対応した理由、第2候補が低予算 e スポーツとなったことから分かる意味検索の限界を考察した。

確認

画像をレポート用の英数字ファイル名で `figures` ディレクトリへ保存し、本文から図番号で参照した後、PDF を再コンパイルした。

対話 7：本文 PDF と付録 PDF の分離

ユーザの指示内容（要約）

「課題の提出形式に従い、レポート本文と付録資料を別々の PDF ファイルに分離する。」

LLM の応答・作業

課題文では本文 PDF と付録 PDF を別ファイルで提出すると指定されていたため、本文末尾に含めていた LLM 利用記録、ソースコード、評価結果の生データを削除し、新しい付録用 TeX ファイルへ移動した。

確認

本文 PDF と付録 PDF を個別にコンパイルし、本文側には指定された五章と参考文献だけが、付録側には補助資料だけが含まれることを確認した。

1.2 LLM 出力の扱い

LLM の提案には、実装環境に存在しないコマンドや、LaTeX 処理系によって利用できない設定が含まれる可能性がある。そのため、生成されたコードや文章を無条件には採用せず、エラーメッセージを確認して修正し、実行結果を根拠として採否を判断した。特に推薦精度 100% という結果については、評価数が 10 件と少なく、開発者が作成したデータであるため、一般的な精度を保証しないことを本文の考察に明記した。

2 ソースコード

作成したソースコードは次のファイルから構成される。

- `app.py` : Streamlit 画面、推薦処理、評価処理
- `pc_builds.csv` : 26 件の PC 構成コーパス
- `requirements.txt` : 実行に必要なライブラリ

実行コマンドを以下に示す。

```
.\venv\Scripts\python.exe -m streamlit run app.py \
--server.fileWatcherType none
```

2.1 アプリケーションのソースコード全文

以下にapp.pyの全文を示す。行番号は説明時の参照を容易にするため付加した。本文では推薦処理の中核だけを掲載し、完全な実装は本付録へ分離した。

```
1 from pathlib import Path
2 import csv
3
4 import pandas as pd
5 import streamlit as st
6 from sentence_transformers import SentenceTransformer, util
7
8
9 DATA_FILE = Path(__file__).with_name("pc_builds.csv")
10 MODEL_NAME = "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
11
12
13 @st.cache_data
14 def load_data() -> list[dict[str, str]]:
15     """ 推薦候補の PC 構成を CSV から読み込む。 """
16     with DATA_FILE.open(encoding="utf-8-sig", newline="") as file:
17         rows = list(csv.DictReader(file))
18
19     required = {
20         "use_case",
21         "price_yen",
22         "cpu",
23         "gpu",
24         "ram",
25         "ssd",
26         "psu",
27         "category",
28         "reason",
29     }
30     if not rows or not required.issubset(rows[0]):
31         raise ValueError("pc_builds.csv の列構成が不正です。")
32     return rows
33
34
35 @st.cache_resource(show_spinner="Sentence-BERT モデルを読み込んでいます…")
36 def load_model() -> SentenceTransformer:
37     return SentenceTransformer(MODEL_NAME)
38
39
40 @st.cache_resource(show_spinner=" 推薦候補をベクトル化しています…")
41 def load_corpus_embeddings():
42     """ 全候補を一度だけベクトル化し、検索のたびの再計算を避ける。 """
43     model = load_model()
44     texts = [row["use_case"] for row in load_data()]
45     return model.encode(
46         texts,
47         convert_to_tensor=True,
48         normalize_embeddings=True,
49     )
50
51
52 def recommend(query: str, budget: int, top_k: int = 3) -> list[dict]:
53     """ 予算内の候補を、入力文とのコサイン類似度順に返す。 """
54     query = query.strip()
55     if not query:
56         return []
57     if budget <= 0 or top_k <= 0:
58         return []
59
60     all_rows = load_data()
61     eligible_indices = [
62         index
63         for index, row in enumerate(all_rows)
64         if int(row["price_yen"]) <= budget
65     ]
66     if not eligible_indices:
67         return []
68
```

```

69 model = load_model()
70 corpus_embeddings = load_corpus_embeddings()
71 query_embedding = model.encode(
72     query,
73     convert_to_tensor=True,
74     normalize_embeddings=True,
75 )
76 eligible_embeddings = corpus_embeddings[eligible_indices]
77 scores = util.cos_sim(query_embedding, eligible_embeddings)[0]
78 top = scores.topk(k=min(top_k, len(eligible_indices)))
79
80 results = []
81 for score, local_index in zip(top.values, top.indices):
82     row = dict(all_rows[eligible_indices[int(local_index)]])
83     row["similarity"] = float(score)
84     row["remaining_budget"] = budget - int(row["price_yen"])
85     results.append(row)
86 return results
87
88
89 TEST_CASES = [
90     ("Apex を高フレームレートで遊びたい", 220_000, "競技 FPS"),
91     ("YouTube 用の動画を編集しながらゲームもしたい", 300_000, "ゲーム+動画編集"),
92     (" ローカル環境で画像生成 AI を動かしたい", 350_000, "生成 AI"),
93     (" 重いゲームを WQHD の高画質で楽しみたい", 280_000, "WQHD ゲーム"),
94     ("4K 解像度で最新ゲームを遊びたい", 360_000, "4K ゲーム"),
95     ("OBS で配信しながら FPS をプレイしたい", 290_000, "ゲーム配信"),
96     ("3DCG 作品のレンダリングを高速化したい", 390_000, "3D 制作"),
97     (" 大学のレポート作成と軽いゲームに使いたい", 160_000, "学生向け"),
98     (" 大規模な街づくりゲームを快適に動かしたい", 270_000, " シミュレーション"),
99     ("VR ヘッドセットでゲームを滑らかに遊びたい", 300_000, "VR ゲーム"),
100 ]
101
102
103 def evaluate() -> tuple[pd.DataFrame, int, int]:
104     """ 固定した 10 件について Top-1/Top-3 カテゴリ正解率を測る。 """
105     rows = []
106     top1_count = 0
107     top3_count = 0
108
109     for text, budget, expected in TEST_CASES:
110         results = recommend(text, budget, 3)
111         predicted = results[0]["category"] if results else " 候補なし "
112         similarity = results[0]["similarity"] if results else 0.0
113         top3 = [row["category"] for row in results]
114
115         top1_ok = predicted == expected
116         top3_ok = expected in top3
117         top1_count += int(top1_ok)
118         top3_count += int(top3_ok)
119
120         rows.append(
121             {
122                 " 入力文 ": text,
123                 " 予算 ": f"{budget:,} 円",
124                 " 期待カテゴリ ": expected,
125                 " 1 位カテゴリ ": predicted,
126                 " 類似度 ": round(similarity, 3),
127                 "Top-1 ": "○" if top1_ok else "×",
128                 "Top-3 ": "○" if top3_ok else "×",
129             }
130         )
131
132     return pd.DataFrame(rows), top1_count, top3_count
133
134
135 st.set_page_config(
136     page_title="PC 構成推薦",
137     page_icon="🖥️",
138     layout="centered",
139 )
140 st.title("🖥️ 用途・予算別 PC 構成推薦")
141 st.caption("Sentence-BERT で入力文の意味を比較し、予算内の構成を推薦します。")

```

```

142
143 search_tab, eval_tab = st.tabs(["構成を検索", "性能評価"])
144
145 with search_tab:
146     query = st.text_area(
147         "どんな用途で使いますか?",
148         "モンハンを WQHD の高画質かつ快適に遊びたい",
149         height=90,
150     )
151     budget_man = st.slider("予算 (万円)", 14, 60, 28)
152     top_k = st.selectbox("表示する候補数", [1, 3, 5], index=1)
153
154     if st.button("おすすめ構成を検索", type="primary", use_container_width=True):
155         if not query.strip():
156             st.warning("用途を入力してください。")
157         else:
158             results = recommend(query, budget_man * 10_000, top_k)
159             if not results:
160                 st.error("予算内の候補がありません。")
161             else:
162                 for rank, row in enumerate(results, 1):
163                     with st.container(border=True):
164                         st.subheader(f"候補 {rank} : {row['category']}")
165                         st.caption(
166                             f"類似度 {row['similarity']:.3f} / "
167                             f"登録用途 : {row['use_case']}"
168                         )
169                         left, right = st.columns(2)
170                         with left:
171                             st.metric("CPU", row["cpu"])
172                             st.metric("RAM", row["ram"])
173                             st.metric("電源", row["psu"])
174                         with right:
175                             st.metric("GPU", row["gpu"])
176                             st.metric("SSD", row["ssd"])
177                             st.metric("概算価格", f"{int(row['price_yen']):,} 円")
178                         st.write(f"** 推薦理由 : ** {row['reason']}")
179                         st.write(f"** 予算残額 : ** {row['remaining_budget']:,} 円")
180
181     with eval_tab:
182         st.subheader("言い換え入力 10 件による一括評価")
183         st.write(
184             "登録文をそのまま使わない 10 件の入力で、"
185             "期待カテゴリが上位候補に含まれるかを評価します。"
186         )
187
188     if st.button("10 件を一括評価", type="primary", use_container_width=True):
189         with st.spinner("評価中です..."):
190             dataframe, top1_count, top3_count = evaluate()
191
192         left, right = st.columns(2)
193         with left:
194             st.metric("Top-1 正解率", f"{top1_count / len(TEST_CASES) * 100:.1f}%")
195         with right:
196             st.metric("Top-3 正解率", f"{top3_count / len(TEST_CASES) * 100:.1f}%")
197
198         st.dataframe(dataframe, use_container_width=True, hide_index=True)
199         st.write(
200             f"Top-1 : {top1_count}/{len(TEST_CASES)} 件、"
201             f"Top-3 : {top3_count}/{len(TEST_CASES)} 件"
202         )

```

3 評価結果の生データ

No.	期待カテゴリ	1 位カテゴリ	Top-3
1	競技 FPS	競技 FPS	正解
2	ゲーム+動画編集	ゲーム+動画編集	正解
3	生成 AI	生成 AI	正解
4	WQHD ゲーム	WQHD ゲーム	正解
5	4K ゲーム	4K ゲーム	正解
6	ゲーム配信	ゲーム配信	正解
7	3D 制作	3D 制作	正解
8	学生向け	学生向け	正解
9	シミュレーション	シミュレーション	正解
10	VR ゲーム	VR ゲーム	正解